



I have this python script

```
#!/usr/bin/env python
#use coherence to calculate how good are clusters - no training - only try to find best number of
topics
#use proper stop words dictionary - add words to stop words, exclude words from stop words.
from future import print_function
import os
import sys
from time import time
#import parse
import load_parsed_htmls
import preprocess_docs
import lemmatization

from sklearn.feature_extraction import text
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.decomposition import NMF, LatentDirichletAllocation
from sklearn.datasets import fetch_20newsgroups
import numpy as np

import pyLDAvis
import pyLDAvis.Ida_model
#import cPickle as pickle
import csv

import nltk
nltk.data.path.append('/home/admin/nltk_data')

from gensim.test.utils import common_corpus, common_dictionary
from gensim.models.coherencemodel import CoherenceModel
from gensim.corpora.dictionary import Dictionary

import cluster_on_word_combinations as prep_docs

dir_path = 'HTML_parsed01/'
n_samples = None #None for all
n_features = 99
n_components = 5
n_top_words = 10
```

```

def print_top_words(model, feature_names, n_top_words):
    outstr = []
    for topic_idx, topic in enumerate(model.components_):
        ascendingIndeces = topic.argsort()[:-n_top_words - 1:-1]
        message = "Topic #%%d: " % topic_idx
        sublist = []
        for idx in ascendingIndeces:
            message += (feature_names[idx] + '\n')
            sublist += [feature_names[idx]]
        outstr += [sublist]
    print(message)
    print()
    return outstr

```

```

def csv_to_string_data(csv_filepath, row_threshold):
    texts = []
    labels = []

```

```

    with open(csv_filepath, newline='', encoding='utf-8') as csvfile:
        reader = csv.reader(csvfile)
        row_count = sum(1 for row in reader)
        if row_threshold is None:
            max_articles = row_count
        else:
            max_articles = row_threshold
        csvfile.seek(0) # Reset file pointer to the beginning
        reader = csv.reader(csvfile) # Create a new reader from the reset file object
        for row_num, row in enumerate(reader, start=1):
            # Skip empty rows or header if any (adapt if needed)
            if not row:
                continue
            if row_num >= max_articles:
                break
            # Example: Combine multiple columns as one document string
            # You can adjust which columns to combine for "document"
            # Here using columns 1 (title) and 3 (description) as text
            doc_text = row[3] + " " + row[1]
            texts.append(doc_text.strip())
            labels.append(row[6]) # Assuming label is in column 6

    return texts, labels

```

```

def get_gensim_coherence(sklearn_lda, tf_feature_names, texts, my_stop_words):
    # tokenized_texts: list of documents, each document is a list of tokens (words)
    topics = []
    for topic_idx, topic_weights in enumerate(sklearn_lda.components_):
        top_word_indices = topic_weights.argsort()[:-n_top_words-1:-1]
        top_words = [tf_feature_names[i] for i in top_word_indices]
        topics.append(top_words)

```

```

#topics_tokenized = [[phrase.split() for phrase in topic] for topic in topics]
topics_tokenized = [
    [token for phrase in topic for token in phrase.split()]
    for topic in topics
]
tokenized_texts = [lemmaToken(doc) for doc in texts]
dictionary = Dictionary(tokenized_texts)
bug_common_corpus = [
    dictionary.doc2bow([token for token in doc_tokens if token not in my_stop_words])
    for doc_tokens in tokenized_texts
]

#print(type(common_corpus)) # should be list or gensim iterable
#print(type(common_corpus[0])) # should be list of tuples
#print(common_corpus[0]) # e.g. [(0, 2), (5, 3)] first doc's bow representation
#for topic in topics:
#    for word in topic:
#        print(word, type(word))

cm = CoherenceModel(topics=topics_tokenized, corpus=bug_common_corpus, dictionary = dicti

coherence = cm.get_coherence() # get coherence value
return coherence

```

```

if name == "main":
if len(sys.argv) < 2:
print('No command-line arguments, assume',dir_path,'for dir path of parsed htmls into jsons.')
else:
dir_path = sys.argv[1]

```

```

#parsed_htmls = load_parsed_htmls.load_parsed_htmls(dir_path, n_samples)
#stringData = preprocess_docs.preprocess_docs(parsed_htmls)
#pickle.dump( stringData, open('stringData.pickle','wb') )
#stringData = pickle.load( open('stringData.pickle','rb') )
all_texts, labels = csv_to_string_data('Issues_Vector21-periodicreview-oct25o.csv', n_san
vectorcast_topics = [
    [ ['build', 'compil'], ['error', 'failure'] ],
    [ ['using source file perspective', 'source file perspective', 'sfp'], ['set', 'using
    [ ['link'], ['error', 'failure'] ],
    [ ['parse', 'environment'], ['error', 'failure'] ],
    [ ['license'], ['error'] ]
]
simple_clusters = prep_docs.process_topics(vectorcast_topics, all_texts, n_samples)
lemmaToken = lemmatization.LemmaTokenizer()

vectorcastStopWords = {'previously', 'previous', 'vectorcast', 'fix', 'version', '2021',
vectorcastUseWords = {'no'}
my_stop_words = list(text.ENGLISH_STOP_WORDS.union(vectorcastStopWords).difference(vector
tfidf_vectorizer = TfidfVectorizer(max_df=0.7,
                                min_df=2,
                                stop_words=my_stop_words,
                                tokenizer = lemmaToken,
                                ngram_range=(2, 4))

```

```

non_clustered_ids = [id for id,cluster in simple_clusters.items() if cluster == -1]
texts = [all_texts[id] for id in non_clustered_ids]
tfidf = tfidf_vectorizer.fit_transform(texts)

print("Fitting LDA models with tf features, "
      "n_samples=%d and n_features=%d..."
      % (len(texts), n_features))

lda = LatentDirichletAllocation(n_components=n_components, max_iter=5,
                               learning_method='online',
                               learning_offset=50.,
                               random_state=0)

t0 = time()
lda.fit(tfidf)
print("done in %0.3fs." % (time() - t0))

print("\nTopics in LDA model:")
tf_feature_names = tfidf_vectorizer.get_feature_names_out()
topwords = print_top_words(lda, tf_feature_names, n_top_words)
print("done in %0.3fs." % (time() - t0))

print("coherence =", get_gensim_coherence(lda, tf_feature_names, texts, my_stop_words))

vis_data = pyLDAvis.lda_model.prepare(lda, tfidf, tfidf_vectorizer)

pyLDAvis.save_html(vis_data, 'lda_vis.html')
#pyLDAvis.show(vis_data)

```

def process_topics(vectorcast_topics, texts, n_samples):

```

if len(sys.argv) > 1:
    csv_filepath = sys.argv[1]

simple_clusters = {}
t_i = 0
for bug_description in texts:
    idx = 0
    for vc_topic in vectorcast_topics:
        res = find_word_combination(vc_topic, bug_description)
        if(res == True):
            simple_clusters[t_i]=idx
            break
    else:
        simple_clusters[t_i]=-1
    idx = idx+1
    if(t_i == 15):
        breakHere = 1
    t_i = t_i + 1

size = len(vectorcast_topics)
print('size = ', len(simple_clusters))
for special_id in range(-1, size):

```

```

count = 0
for doc_idx, cluster_id in simple_clusters.items(): # Proper iteration
    if cluster_id == special_id:
        print(f"Document {doc_idx}: Cluster {cluster_id} - Text Sample: {texts[doc_idx]}")
        count += 1
if count == 0:
    print(f"No documents in cluster {special_id}")

return simple_clusters

```

Currently

```
vis_data = pyLDavis.Lda_model.prepare(lda, tfidf, tfidf_vectorizer)
```

```
pyLDavis.save_html(vis_data, 'lda_vis.html')
```

only creates results for the second clustering with lda. I would like to view all clusters together and possibly the top words for all clusters including the created clusters with

```
simple_clusters = prep_docs.process_topics(vectorcast_topics, all_texts, n_samples)
```

To visualize all clusters together (both the rule-based `simple_clusters` from `vectorcast_topics` and the LDA topics), modify your script to create a combined topic representation and generate a single pyLDavis visualization. This involves extracting top words for the rule-based clusters, merging them with LDA topics, and preparing a custom visualization data structure that pyLDavis can handle for both cluster types.^{[1] [2]}

Extract Top Words for Rule-Based Clusters

Add this function after `print_top_words` to get top words for `simple_clusters`:

```

def get_rule_cluster_top_words(vectorcast_topics, tf_feature_names, texts, lemmaToken, my_vocab):
    rule_topics = []
    for topic_idx, vc_topic in enumerate(vectorcast_topics):
        # Flatten word combinations into top words for this cluster
        flat_words = [word for combo in vc_topic for word in combo]
        # Optionally score/filter by TF-IDF relevance from your vectorizer
        relevant_words = [w for w in flat_words if w in tf_feature_names]
        rule_topics.append(relevant_words[:n_top_words]) # Limit to n_top_words
        print(f"Rule Cluster #{topic_idx}: {relevant_words[:n_top_words]}")
    return rule_topics

```

Combine and Visualize All Clusters

Replace your current visualization block with this (call it after fitting LDA and printing LDA topics):

```

# Get top words for rule-based clusters (including unassigned as a special "cluster")
rule_topics = get_rule_cluster_top_words(vectorcast_topics, tf_feature_names, all_texts,
# Append LDA topics (they're already extracted as topwords)
all_topics = rule_topics + topwords # topwords from print_top_words(lda, ...)

```

```

# Tokenize all topics for coherence (optional, but matches your existing pattern)
all_topics_tokenized = [[token for phrase in topic for token in phrase.split()] for topic in topics]

# Custom pyLDAvis prep for combined topics (use LDA data as base, override topics)
# Note: pyLDAvis expects scikit-learn LDA; for rule clusters, we approximate with dummy corpus
from sklearn.feature_extraction.text import CountVectorizer
dummy_cv = CountVectorizer(max_features=n_features, tokenizer=lemmaToken, stop_words=my_stop_words)
dummy_corpus = dummy_cv.fit_transform(all_texts)

# Prepare with combined topic info (LDA for LDA part, rule words for rule part)
vis_data = pyLDAvis.lda_model.prepare(
    lda, tfidf, tfidf_vectorizer,
    mds='tsne', # Or 'mmds' for faster
    topics=all_topics_tokenized[:n_components * 2] # Adjust length to total clusters
)

pyLDAvis.save_html(vis_data, 'all_clusters_vis.html')
print("Combined visualization saved to 'all_clusters_vis.html'")

```

Usage and Printing All Top Words

Update your main block to call the new function and print everything:

```

# After simple_clusters = prep_docs.process_topics(...)
print("Rule-based clusters top words:")
rule_topwords = get_rule_cluster_top_words(vectorcast_topics, tf_feature_names, all_texts)

# After LDA fitting...
print("LDA topics top words:")
lda_topwords = print_top_words(lda, tf_feature_names, n_top_words)

print("All clusters coherence (approx):", get_gensim_coherence(lda, tf_feature_names, texts))

```

This creates `all_clusters_vis.html` showing ~10 circles (5 rule + 5 LDA), with inter-topic distances and top words from both methods. Rule clusters use their predefined word combos as "topics"; tune `n_top_words` or add TF-IDF weighting for better relevance. For true hybrid coherence across all, compute separately and average. Run on non-clustered texts only, as before. [\[3\]](#) [\[4\]](#)

✱

After your changes I got

File `/usr/lib/python3/dist-packages/spyder_kernels/py3compat.py:356` in `compat_exec`
`exec(code, globals, locals)`

File `~/workspace/sunnystronghold/run.py:188`
`all_topics = rule_topics + topwords # topwords from print_top_words(lda, ...)`

`NameError: name 'topwords' is not defined`

The error "NameError: name 'topwords' is not defined" means that the variable `topwords` was referenced before it was assigned a value in your script. This likely happened because the variable returned by `print_top_words(lda, tf_feature_names, n_top_words)` was not assigned to `topwords` before it was used in the line `all_topics = rule_topics + topwords`.

To fix this, ensure you capture the return value of your `print_top_words` function in a variable named `topwords` before using it. For example, modify your script like this:

```
# After fitting the LDA model
lda = LatentDirichletAllocation(n_components=n_components, max_iter=5,
                               learning_method='online',
                               learning_offset=50.,
                               random_state=0)

lda.fit(tfidf)

tf_feature_names = tfidf_vectorizer.get_feature_names_out()

# Capture the returned top words in 'topwords'
topwords = print_top_words(lda, tf_feature_names, n_top_words)

# Then combine with rule-based topics
rule_topics = get_rule_cluster_top_words(vectorcast_topics, tf_feature_names, all_texts,

all_topics = rule_topics + topwords
```

Make sure `topwords` is defined exactly as shown before it is used. This will resolve the `NameError` by ensuring the variable exists in the current scope when you concatenate the topic lists.

It's a common Python error that happens when you forget to assign the output of a function call or reference a variable before its assignment. [\[11\]](#) [\[12\]](#)

✱✱

I got the following output + error

All clusters coherence (approx): -6.100170538361094

done in 2.016s.

coherence = -6.100170538361094

Rule Cluster #0: []

Rule Cluster #1: ['source file perspective']

Rule Cluster #2: []

Rule Cluster #3: []

Rule Cluster #4: []

/home/admin/.local/lib/python3.11/site-packages/sklearn/feature_extraction/text.py:517:

UserWarning: The parameter 'token_pattern' will not be used since 'tokenizer' is not None'

warnings.warn(

/home/admin/.local/lib/python3.11/site-packages/sklearn/feature_extraction/text.py:402:

UserWarning: Your stop_words may be inconsistent with your preprocessing. Tokenizing the stop words generated tokens ['far', 'make'] not in stop_words.

```
warnings.warn(
```

```
Traceback (most recent call last):
```

```
File /usr/lib/python3/dist-packages/spyder_kernels/py3compat.py:356 in compat_exec  
exec(code, globals, locals)
```

```
File ~/workspace/sunnystronghold/run.py:200
```

```
vis_data = pyLDAvis.lda_model.prepare(
```

```
File /usr/local/lib/python3.11/dist-packages/pyLDAvis/lda_model.py:95 in prepare
```

```
return pyLDAvis.prepare(**opts)
```

```
TypeError: prepare() got an unexpected keyword argument 'topics'
```

The error you encountered,

```
TypeError: prepare() got an unexpected keyword argument 'topics'
```

is because the `pyLDAvis.lda_model.prepare` function does not support a `topics` keyword argument. The `prepare` function provided by `pyLDAvis` only accepts these main parameters: the fitted LDA model object, the document-term matrix (corpus), and the vectorizer, along with optional visualization settings like MDS methods, but it does not allow manual injection of a combined topics list or overriding the topics directly.

What this means for combining your clusters visualization

- `pyLDAvis` is designed to visualize one LDA model's output at a time (a single topic model).
- You cannot pass combined or custom topic lists directly to `prepare`.
- Attempting to visualize both your rule-based clusters and LDA clusters together in one `pyLDAvis` visualization is not supported out-of-the-box.

Workarounds and recommendations

1. Visualize separately:

- Generate your current `lda_vis.html` for the LDA topics.
- Build a separate visualization or display for your rule-based `simple_clusters`. For example, print out the top words per cluster or create a manual HTML page or bar plots showing cluster compositions.

2. Merge textual data before modeling:

- Instead of combining topics after modeling, consider representing your rule-based clusters as additional "documents" or features and then run LDA to discover unified topics.

3. Custom visualization:

- If you want an integrated view, you need to implement a custom visualization (e.g., using `matplotlib` or other visualization tools) that overlays both cluster sets side by side,

showing their top words and document counts.

4. Warnings to address:

- The warning about `token_pattern` is typical when using custom tokenizers with scikit-learn vectorizers and can often be ignored.
- The stop words warning indicates some tokens derived from stop words don't exactly match your stop word list; you might want to review and harmonize your stop words and tokenizer preprocessing.

Summary

You must remove the attempt to pass `topics=all_topics_tokenized` to `pyLDAvis.lda_model.prepare`. Instead, generate separate visualizations or reports for your rule-based clusters and the LDA topics. `pyLDAvis` does not support visualizing multiple different clusterings or manually combined topic sets in one plot.

```
# Correct usage for LDA visualization only
vis_data = pyLDAvis.lda_model.prepare(lda, tfidf, tfidf_vectorizer)
pyLDAvis.save_html(vis_data, 'lda_vis.html')
```

For your rule-based clusters, consider simply printing top words as your script already does, or create custom charts separately.

This aligns with `pyLDAvis`'s design, which visualizes a single topic model's learned distribution, not an arbitrary set of clusters with manual topic lists. [\[21\]](#) [\[22\]](#) [\[23\]](#)

✱

1. https://markroxxor.github.io/gensim/static/notebooks/topic_coherence_tutorial.html
2. https://radimrehurek.com/gensim_3.8.3/models/coherencemodel.html
3. <https://tedboy.github.io/nlps/generated/generated/gensim.models.CoherenceModel.html>
4. <https://radimrehurek.com/gensim/models/coherencemodel.html>
5. https://notebook.community/sridhar-newsdistill/Data-Science-45min-Intros/topic-modeling-101/topic_modeling_part1
6. <https://learn.scds.ca/text-analysis-3/lessons/tmpython.html>
7. <https://elibooklover.github.io/Tutorials/Python/topicmodelingLDA/>
8. <https://teemukanstren.com/2017/05/28/topicmodels-topicmodels/>
9. https://github.com/piskvorky/gensim/blob/develop/gensim/topic_coherence/text_analysis.py
10. <https://stackoverflow.com/questions/60246570/gensim-lda-coherence-score-nan>
11. <https://www.geeksforgeeks.org/python/how-to-fix-variable-not-defined-in-custom-modules-in-python/>
12. <https://www.geeksforgeeks.org/python/undefined-variable-nameerror-in-python/>
13. <https://labex.io/tutorials/python-how-to-handle-nameerror-with-undefined-variables-in-python-417773>
14. <https://www.freecodecamp.org/news/nameerror-name-not-defined-how-to-fix-this-python-error/>

15. <https://rollbar.com/blog/undefined-variable-nameerror-python/>
16. <https://stackoverflow.com/questions/37955081/python-nameerror-variable-not-defined>
17. <https://discuss.python.org/t/nameerror-name-is-not-defined-but-it-is/11535>
18. https://www.reddit.com/r/learnpython/comments/13w3g9c/nameerror_when_variable_has_been_defined/
19. <https://www.youtube.com/watch?v=nomaHDS2LnA>
20. <https://stackoverflow.com/questions/73871606/nameerror-name-variable-is-not-defined-how-to-handle-and-exception-in-the-fil>
21. https://blog.csdn.net/m0_57265868/article/details/139445498
22. <https://github.com/bmabey/pyLDavis/issues/208>
23. <https://github.com/bmabey/pyLDavis/issues/251>
24. https://www.reddit.com/r/learnpython/comments/ft58g8/visualizing_lda_model_with_pyldavis/
25. https://pyldavis.readthedocs.io/en/latest/_modules/pyLDavis/_prepare.html
26. <https://github.com/bmabey/pyLDavis/issues/225>
27. <https://stackoverflow.com/questions/75397231/pyldavis-error-attributeerror-countvectorizer-object-has-no-attribute-get-fe>
28. <https://github.com/bmabey/pyLDavis/issues/245>
29. <https://stackoverflow.com/questions/tagged/pyldavis>
30. https://github.com/bmabey/pyLDavis/blob/master/pyLDavis/_prepare.py